

UNIVERSIDAD TECNOLÓGICA DE LOS ANDES



Facultad de Ingeniería de Sistemas

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS E INFORMATICA



TRABAJO MONOGRÁFICO

TRANSFORMACIÓN 3D

Monografía presentada para el curso de Computación Gráfica

Autor: Eliseo Pancorbo Diaz

Docente: Elvio Tintaya Zegarra

Cusco, Perú — 2026

RESUMEN

Las transformaciones 3D son operaciones matemáticas fundamentales dentro de la computación gráfica que permiten modificar la posición, orientación, escala y perspectiva de los objetos en un espacio tridimensional. Su correcto entendimiento y aplicación resulta indispensable para el desarrollo de aplicaciones gráficas interactivas, motores de videojuegos, simuladores y sistemas de visualización científica. La presente monografía describe de manera sistemática las bases matemáticas de las transformaciones 3D —haciendo énfasis en el uso de matrices homogéneas y cuaterniones—, sus tipos principales (traslación, rotación, escalado, cizallamiento y proyección), su implementación en el pipeline gráfico moderno y sus aplicaciones en la industria contemporánea.

1. INTRODUCCIÓN

La representación y manipulación de objetos en un espacio tridimensional constituye uno de los problemas centrales de la computación gráfica. Para que un motor gráfico pueda mostrar una escena en movimiento —un personaje que camina, una cámara que orbita un edificio o un robot industrial que articula sus segmentos— es necesario aplicar de forma continua y eficiente operaciones matemáticas sobre la geometría de cada objeto. Estas operaciones se denominan transformaciones geométricas 3D (Foley et al., 1996).

El matemático Arthur Cayley sentó las bases del álgebra matricial en el siglo XIX, herramienta que décadas después sería adoptada por la comunidad de computación gráfica como el formalismo estándar para representar y combinar transformaciones. En 1965, Ivan Sutherland propuso en su tesis doctoral el uso de matrices 4×4 en coordenadas homogéneas para unificar traslación, rotación y escalado en una sola estructura algebraica (Shirley y Marschner, 2009). Este enfoque sigue vigente en todas las APIs y motores gráficos modernos.

El presente trabajo monográfico aborda los fundamentos matemáticos de las transformaciones 3D, los tipos de transformaciones más relevantes, el papel de los cuaterniones en la

representación de rotaciones, su integración en el pipeline gráfico y sus aplicaciones prácticas en la industria del software.

2. FUNDAMENTOS MATEMÁTICOS

2.1 El espacio tridimensional y los sistemas de coordenadas

En computación gráfica, el espacio tridimensional se modela mediante un sistema de coordenadas cartesiano con tres ejes ortogonales: X, Y y Z. Existen dos convenciones principales para la orientación de dichos ejes: el sistema de mano derecha (utilizado por OpenGL, Blender y la mayoría de entornos académicos) y el sistema de mano izquierda (empleado por DirectX y Unity) (Akenine-Möller et al., 2018). La elección de la convención afecta el signo de algunas operaciones de rotación y producto vectorial, por lo que es fundamental conocer cuál utiliza el sistema con el que se trabaja.

Un punto en el espacio 3D se representa como un vector columna $P = (x, y, z)^T$. Para poder expresar la traslación como una multiplicación matricial —junto con rotación y escalado—, se introduce una cuarta coordenada w , obteniendo el vector homogéneo $\tilde{P} = (x, y, z, w)^T$. Cuando $w = 1$, el vector representa un punto; cuando $w = 0$, representa una dirección o vector libre (Foley et al., 1996).

2.2 Matrices de transformación homogéneas 4×4

La gran ventaja de las coordenadas homogéneas es que todas las transformaciones afines —traslación, rotación, escalado y cizallamiento— pueden representarse como matrices 4×4 y combinarse mediante simple multiplicación matricial. Esto permite al hardware gráfico (GPU) aplicar en paralelo y de forma extremadamente eficiente una cadena de transformaciones a millones de vértices por segundo (Sellers y Kessenich, 2016).

La forma general de una matriz de transformación homogénea es:

$$M = \begin{bmatrix} R & | & t \end{bmatrix} \text{ donde } R \text{ es la submatriz } 3 \times 3 \text{ de rotación/escala y } t \text{ es el vector de traslación } 3 \times 1$$

La última fila $[0, 0, 0, 1]$ preserva la coordenada homogénea w y garantiza que la concatenación de transformaciones afines produzca siempre otra transformación afín (Foley et al., 1996).

Aplicar n transformaciones concatenadas a un vértice requiere una sola multiplicación $M = M_n \cdot \dots \cdot M_2 \cdot M_1$, lo que reduce enormemente la carga computacional en tiempo de ejecución.

3. TIPOS DE TRANSFORMACIONES 3D

3.1 Traslación

La traslación desplaza un objeto una distancia (t_x, t_y, t_z) a lo largo de los ejes X, Y y Z respectivamente. En coordenadas homogéneas, la matriz de traslación $T(t_x, t_y, t_z)$ es una matriz identidad 4×4 con los valores de desplazamiento en la última columna de las tres primeras filas (Shirley y Marschner, 2009):

$$T = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & t_x & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & t_y & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & t_z & 0 & 0 & 0 & 1 \end{array} \right]$$

La traslación es una transformación no lineal en el espacio euclidiano, pero se vuelve lineal (y por tanto matricial) gracias a las coordenadas homogéneas. Esta es precisamente la razón por la que estas coordenadas son tan valiosas en computación gráfica (Akenine-Möller et al., 2018).

3.2 Rotación

La rotación gira un objeto un ángulo θ alrededor de uno de los ejes coordenados o alrededor de un eje arbitrario. Las matrices de rotación básicas alrededor de los ejes X, Y y Z son matrices ortogonales 3×3 (embebidas en la submatriz superior izquierda de la matriz 4×4) cuya transpuesta es igual a su inversa, propiedad que garantiza que la rotación preserve distancias y ángulos (Foley et al., 1996).

La rotación alrededor de un eje arbitrario unitario $\hat{u} = (u_x, u_y, u_z)$ por un ángulo θ puede calcularse mediante la fórmula de Rodrigues, que expresa la matriz de rotación en función de los componentes del eje y las funciones trigonométricas del ángulo (Shirley y Marschner, 2009). Esta fórmula es la base de muchas implementaciones prácticas de rotación en motores gráficos.

Un problema clásico al componer tres rotaciones sucesivas alrededor de los ejes coordenados —conocido como ángulos de Euler— es el bloqueo de cardán (gimbal lock): situación en la que dos ejes de rotación quedan alineados y el sistema pierde un grado de libertad. Para evitar

este problema, la industria gráfica emplea cuaterniones (Shoemake, 1985, citado en Akenine-Möller et al., 2018).

3.3 Escalado

El escalado modifica las dimensiones de un objeto multiplicando sus coordenadas por factores de escala (s_x , s_y , s_z). Si los tres factores son iguales se denomina escalado uniforme o isótropo; en caso contrario, escalado no uniforme o anisótropo. El escalado uniforme preserva la forma del objeto y las relaciones angulares entre sus superficies, mientras que el no uniforme puede deformar el objeto (Gonzalez y Woods, 2008).

Un caso especial es el escalado con factor negativo, que produce una reflexión del objeto respecto al plano correspondiente. Por ejemplo, un factor $s_x = -1$ refleja el objeto respecto al plano YZ, equivalente a un espejo vertical.

3.4 Cizallamiento (Shear)

El cizallamiento (shear) es una transformación que inclina la geometría de un objeto desplazando sus puntos en una dirección proporcional a su coordenada en otra dirección. Aunque menos frecuente que las transformaciones anteriores, el cizallamiento se utiliza para simular efectos de perspectiva oblicua, sombras proyectadas y deformaciones de malla en animación (Akenine-Möller et al., 2018). En la matriz homogénea, los términos fuera de la diagonal de la submatriz 3×3 codifican el cizallamiento.

3.5 Transformación de proyección

La proyección transforma el espacio tridimensional en una imagen bidimensional, simulando la manera en que un observador o una cámara percibe la escena. Existen dos tipos principales:

- Proyección ortogonal (o paralela): los rayos de proyección son paralelos entre sí y perpendiculares al plano de proyección. Se utiliza en aplicaciones de CAD, planos arquitectónicos y vistas técnicas, ya que preserva las dimensiones relativas de los objetos independientemente de su distancia al observador (Foley et al., 1996).
- Proyección perspectiva: simula el efecto de perspectiva del ojo humano, en el que los objetos más lejanos aparecen más pequeños. Se define mediante el centro de proyección (posición de la cámara) y el plano de imagen. La matriz de perspectiva

transforma el frustum de visión en un cubo normalizado (NDC, Normalized Device Coordinates) listo para la rasterización (Shirley y Marschner, 2009).

4. CUATERNIONES EN LA REPRESENTACIÓN DE ROTACIONES

4.1 Definición y propiedades

Los cuaterniones son una extensión de los números complejos al espacio de cuatro dimensiones, propuestos por el matemático irlandés William Rowan Hamilton en 1843. Un cuaternión se define como $q = w + xi + yj + zk$, donde w es la parte escalar, (x, y, z) es la parte vectorial e i, j, k son unidades imaginarias que satisfacen $i^2 = j^2 = k^2 = ijk = -1$ (Kuipers, 2002).

Un cuaternión unitario (con norma igual a 1) representa una rotación 3D de ángulo θ alrededor del eje unitario $\hat{u}$ mediante la expresión $q = \cos(\theta/2) + \sin(\theta/2) \cdot \hat{u}$. Para rotar un punto P mediante el cuaternión q , la operación es $P' = q \cdot P \cdot q^*$, donde q^* es el conjugado del cuaternión (Hanson, 2006).

4.2 Ventajas frente a las matrices de rotación y los ángulos de Euler

Los cuaterniones presentan ventajas significativas respecto a las matrices y los ángulos de Euler para representar rotaciones 3D en aplicaciones gráficas interactivas (Kuipers, 2002):

- Evitan completamente el bloqueo de cardán (gimbal lock) que afecta a los ángulos de Euler.
- Permiten la interpolación suave entre dos rotaciones mediante SLERP (Spherical Linear Interpolation), produciendo animaciones de rotación naturales y continuas sin artefactos visuales.
- Son computacionalmente más eficientes que las matrices 4×4 para concatenar rotaciones: requieren 16 multiplicaciones frente a las 64 de la multiplicación matricial completa.
- Ocupan menos memoria: 4 valores flotantes frente a los 16 de una matriz 4×4 .

Por estas razones, motores gráficos como Unity, Unreal Engine y Godot utilizan cuaterniones como representación interna de las rotaciones de los objetos (Epic Games, 2023; Unity Technologies, 2023).

4.3 Interpolación SLERP

La interpolación esférica lineal (SLERP, Spherical Linear Interpolation) entre dos cuaterniones q_1 y q_2 con parámetro $t \in [0,1]$ calcula el cuaternión que representa la rotación intermedia recorriendo la geodésica de la hipersfera unitaria en el espacio de cuaterniones. Esto garantiza una velocidad angular constante durante la interpolación, a diferencia de la interpolación lineal simple (LERP) que produce aceleraciones y desaceleraciones no deseadas (Shoemake, 1985, citado en Akenine-Möller et al., 2018).

5. CONCATENACIÓN DE TRANSFORMACIONES Y JERARQUÍAS

5.1 Concatenación de matrices

Una de las propiedades más poderosas de la representación matricial de transformaciones es que la composición de múltiples transformaciones puede precomputarse como una sola matriz producto. Si se desea aplicar primero una transformación T_1 y luego T_2 , la transformación compuesta es $M = T_2 \cdot T_1$ (nótese el orden inverso, consecuencia de la no conmutatividad del producto matricial). Esta propiedad permite al motor gráfico calcular la matriz de transformación total de un objeto una sola vez por fotograma y aplicarla a todos sus vértices con una sola multiplicación (Foley et al., 1996).

Es importante señalar que la multiplicación matricial no es conmutativa: en general, $T_2 \cdot T_1 \neq T_1 \cdot T_2$. Un error frecuente en el desarrollo de aplicaciones gráficas es invertir el orden de las transformaciones, produciendo resultados visuales incorrectos (Shirley y Marschner, 2009).

5.2 Grafo de escena y transformaciones jerárquicas

Las aplicaciones gráficas complejas organizan los objetos de la escena en una estructura jerárquica denominada grafo de escena (scene graph). En esta estructura, cada nodo puede tener una transformación local que se aplica en relación con su nodo padre. La transformación

global de un nodo se obtiene multiplicando su transformación local por las transformaciones de todos sus antecesores en la jerarquía (Akenine-Möller et al., 2018).

Este enfoque jerárquico es fundamental para modelar sistemas articulados como esqueletos de personajes, brazos robóticos o jerarquías de componentes de una nave espacial. Por ejemplo, al rotar el hombro de un personaje, el codo, la muñeca y la mano deben heredar automáticamente esa rotación, lo que se logra de forma natural con la propagación de transformaciones en el grafo de escena (Unity Technologies, 2023).

5.3 Sistemas de coordenadas local y global

En el desarrollo de aplicaciones 3D es fundamental distinguir entre el sistema de coordenadas local de cada objeto (también llamado espacio objeto) y el sistema de coordenadas global de la escena (espacio mundo). La matriz de modelo (model matrix) de cada objeto describe precisamente la transformación que lleva su geometría del espacio objeto al espacio mundo. El flujo completo de transformaciones en el pipeline estándar es: espacio objeto $\rightarrow$ espacio mundo (matriz de modelo) $\rightarrow$ espacio cámara (matriz de vista) $\rightarrow$ espacio recortado (matriz de proyección) $\rightarrow$ espacio de pantalla (división por w y viewport transform) (Sellers y Kessenich, 2016).

6. IMPLEMENTACIÓN EN EL PIPELINE GRÁFICO MODERNO

6.1 Vertex shaders y transformaciones en la GPU

En el pipeline gráfico programable moderno (OpenGL, Vulkan, DirectX 12), las transformaciones geométricas se implementan en el vertex shader, un pequeño programa que se ejecuta en paralelo en la GPU para cada vértice de la malla. El vertex shader recibe la posición del vértice en espacio objeto y las matrices de transformación como variables uniformes, calcula la posición final en espacio de recorte y puede transmitir atributos adicionales (normales, coordenadas de textura, color) a las etapas posteriores del pipeline (Rost et al., 2009).

La eficiencia de este proceso es extraordinaria: una GPU de gama media puede procesar cientos de millones de vértices por segundo, cada uno sometido a multiplicaciones de matrices 4×4 ,

gracias a la arquitectura masivamente paralela de las unidades de procesamiento de sombreado (SIMD).

6.2 Transformación de normales

Las normales de superficie son vectores perpendiculares a la geometría del objeto y resultan fundamentales para los cálculos de iluminación. Sin embargo, las normales no se transforman de la misma manera que los vértices: en presencia de escalados no uniformes, transformar las normales con la misma matriz que los vértices produce normales incorrectas que no son perpendiculares a la superficie transformada. La transformación correcta de normales utiliza la transpuesta de la inversa de la submatriz 3×3 de la matriz de modelo (Foley et al., 1996). Este detalle, frecuentemente pasado por alto por principiantes, es crítico para obtener una iluminación visualmente correcta.

6.3 Bibliotecas matemáticas: GLM

En la práctica, la implementación manual de todas las matrices de transformación es tediosa y propensa a errores. Por ello, la comunidad de desarrolladores utiliza bibliotecas matemáticas especializadas. GLM (OpenGL Mathematics) es la biblioteca estándar de facto para aplicaciones C++/OpenGL: proporciona implementaciones eficientes y correctas de vectores, matrices, cuaterniones y todas las transformaciones descritas en este trabajo, con una sintaxis que imita directamente la del lenguaje de shaders GLSL (Grobler, 2014). Otras opciones populares son Eigen (para aplicaciones científicas y robótica) y DirectXMath (para el ecosistema Microsoft).

7. APLICACIONES PRÁCTICAS

7.1 Animación de personajes y skinning

La animación de personajes 3D se basa en un sistema de esqueleto y piel (skeleton-skin o skinning). El esqueleto es una jerarquía de huesos, cada uno con su propia transformación local (típicamente representada como cuaternión + traslación). Cada vértice de la malla del personaje está influenciado por uno o varios huesos con pesos asociados. En cada fotograma de la animación, las transformaciones de los huesos se actualizan, se propagan por el grafo jerárquico y se aplican a los vértices mediante una suma ponderada de transformaciones

(Linear Blend Skinning), produciendo el movimiento fluido del personaje (Akenine-Möller et al., 2018).

7.2 Sistemas de cámara

La cámara virtual en un sistema 3D se modela como un objeto con su propia transformación en el espacio mundo. La matriz de vista (view matrix) es la inversa de la transformación de la cámara: lleva todos los objetos de la escena al sistema de referencia de la cámara. Técnicas como la cámara en primera persona, la cámara orbital y el seguimiento de objetivos (look-at) se implementan construyendo dinámicamente esta matriz de vista a partir de la posición, el punto de interés y el vector arriba de la cámara (Shirley y Marschner, 2009).

7.3 Simulación y realidad virtual

En simuladores de vuelo, conducción y entrenamiento industrial, las transformaciones 3D se aplican en tiempo real para sincronizar la vista renderizada con los movimientos del operador. En sistemas de realidad virtual, las matrices de vista de cada ojo se actualizan a frecuencias de 90 Hz o superiores a partir de los datos del sensor de orientación del casco (giróscopo + acelerómetro), fusionados típicamente mediante filtros de Kalman o complementarios que producen cuaterniones de orientación con latencia mínima (Aukstakalnis, 2017).

8. CONCLUSIONES

Las transformaciones 3D constituyen el núcleo matemático de la computación gráfica moderna. La representación mediante matrices homogéneas 4×4 proporciona un marco algebraico unificado que permite expresar traslación, rotación, escalado y proyección con una misma estructura, facilitando su concatenación eficiente en el hardware de la GPU.

Los cuaterniones ofrecen ventajas sustanciales sobre las matrices y los ángulos de Euler para la representación e interpolación de rotaciones, eliminando el bloqueo de cardán y habilitando la interpolación suave mediante SLERP. Por estas razones son el estándar en los motores gráficos industriales.

La organización jerárquica de transformaciones en grafos de escena y el flujo de espacios de coordenadas —objeto, mundo, cámara, recorte y pantalla— son conocimientos fundamentales

para cualquier desarrollador de aplicaciones gráficas 3D, desde videojuegos hasta simuladores industriales y sistemas de realidad virtual.

El dominio de estos conceptos, combinado con el uso de bibliotecas matemáticas como GLM y la comprensión del pipeline gráfico programable, habilita al estudiante para abordar proyectos de computación gráfica de complejidad creciente con bases teóricas sólidas y herramientas prácticas efectivas.

REFERENCIAS BIBLIOGRÁFICAS

- Akenine-Möller, T., Haines, E. y Hoffman, N. (2018). Real-Time Rendering (4.^a ed.). CRC Press.
- Aukstakalnis, S. (2017). Practical Augmented Reality: A Guide to the Technologies, Applications, and Human Factors for AR and VR. Addison-Wesley Professional.
- Epic Games. (2023). Unreal Engine 5: Rotator y transformaciones de actores. Documentación oficial. <https://docs.unrealengine.com/5.0/en-US/transforming-actors-in-unreal-engine/>
- Foley, J. D., Van Dam, A., Feiner, S. K. y Hughes, J. F. (1996). Computer Graphics: Principles and Practice (2.^a ed. en C). Addison-Wesley Iberoamericana.
- Gonzalez, R. C. y Woods, R. E. (2008). Procesamiento digital de imágenes (3.^a ed.). Pearson Educación.
- Grobler, G. (2014). GLM: OpenGL Mathematics library for C++. <https://github.com/g-truc/glm>
- Hanson, A. J. (2006). Visualizing Quaternions. Morgan Kaufmann / Elsevier.
- Kuipers, J. B. (2002). Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality. Princeton University Press.
- Rost, R. J., Licea-Kane, B., Ginsburg, D. y Kessenich, J. M. (2009). OpenGL Shading Language (3.^a ed.). Addison-Wesley Professional.
- Sellers, G. y Kessenich, J. (2016). Vulkan Programming Guide: The Official Guide to Learning Vulkan. Addison-Wesley Professional.

Shirley, P. y Marschner, S. (2009). Fundamentals of Computer Graphics (3.^a ed.). CRC Press.

Unity Technologies. (2023). Transforms — Unity Manual.
<https://docs.unity3d.com/Manual/class-Transform.html>